

MODULES 31 – 43

LangChain Projects, RAG Apps & Deployment

- Module 31 — Chatbots with Conversation History
- Module 32 — Conversational Q&A ChatBot
- Module 33 — End-to-End Q&A ChatBot Gen AI App
- Module 34 — RAG Document Q&A with Groq API and LLaMA 3
- Module 35 — Conversational Q&A Chat with PDF ChatBot
- Module 37 — Chat with SQL DB using LangChain SQL Toolkit
- Module 38 — Text Summarization with LangChain
- Module 39 — Text to Math Problem Solver
- Module 40 — HuggingFace and LangChain Integration
- Module 41 — PDF Query RAG with LangChain and AstraDB
- Module 42 — Working with CodeLlama via Ollama
- Module 43 — Deployment of Gen AI Apps

Module 31 Building Chatbots with Conversation History

Build multi-turn chatbots in LangChain using message history and prompt templates, then upgrade to a full RAG-powered Q&A chatbot that remembers past turns.

Lectures 1 & 2 — Message History and MessagesPlaceholder

```
from langchain_openai import ChatOpenAI
from langchain_core.prompts import ChatPromptTemplate, MessagesPlaceholder
from langchain_core.chat_history import InMemoryChatMessageHistory
from langchain_core.runnables.history import RunnableWithMessageHistory
from langchain_core.output_parsers import StrOutputParser

llm = ChatOpenAI(model='gpt-4o')
store = {}

def get_history(session_id: str):
    if session_id not in store:
        store[session_id] = InMemoryChatMessageHistory()
    return store[session_id]

prompt = ChatPromptTemplate.from_messages([
    ('system', 'You are a helpful assistant.'),
    MessagesPlaceholder('history'), # injected at runtime
    ('human', '{input}'))
])

chain = prompt | llm | StrOutputParser()
chatbot = RunnableWithMessageHistory(chain, get_history,
    input_messages_key='input', history_messages_key='history')

cfg = {'configurable': {'session_id': 'ul'}}
chatbot.invoke({'input': 'My name is Alice'}, config=cfg)
print(chatbot.invoke({'input': 'What is my name?'}, config=cfg))
```

Lecture 3 — Trimming Chat History

```
from langchain_core.messages import trim_messages

# Keep only last 10 messages to stay within context window
trimmer = trim_messages(max_tokens=1000, strategy='last',
    token_counter=llm, include_system=True)

trimmed_prompt = ChatPromptTemplate.from_messages([
    ('system', 'You are a helpful assistant.'),
    MessagesPlaceholder('history'),
    ('human', '{input}'))
])

# Add trimmer before the prompt in the chain
from langchain_core.runnables import RunnablePassthrough
chain = (
    RunnablePassthrough.assign(history=lambda x: trimmer.invoke(x['history']))
    | trimmed_prompt | llm | StrOutputParser()
)
```

Lecture 4 — RAG Chatbot with Message History

```
# Combine retriever + memory in one chain
from langchain.chains import create_retrieval_chain
from langchain.chains.combine_documents import create_stuff_documents_chain
from langchain.chains import create_history_aware_retriever

# Step 1: History-aware retriever reformulates query with context
history_aware_retriever = create_history_aware_retriever(
    llm, retriever, contextualize_q_prompt)

# Step 2: Stuff docs + answer prompt
```

```
qa_chain = create_stuff_documents_chain(llm, qa_prompt)

# Step 3: Full RAG chain
rag_chain = create_retrieval_chain(history_aware_retriever, qa_chain)
with_memory = RunnableWithMessageHistory(rag_chain, get_history,
    input_messages_key='input', history_messages_key='chat_history',
    output_messages_key='answer')
```

Module 32 Conversational Q&A ChatBot with Message History

A focused single-lecture module that implements the complete conversational RAG chatbot — end-to-end from document indexing to multi-turn Q&A with history.

Lecture 1 — Q&A RAG Chatbot with Message History

```
from langchain_community.document_loaders import WebBaseLoader
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain_openai import OpenAIEmbeddings, ChatOpenAI
from langchain_community.vectorstores import Chroma
from langchain.chains import create_retrieval_chain, create_history_aware_retriever
from langchain.chains.combine_documents import create_stuff_documents_chain
from langchain_core.prompts import ChatPromptTemplate, MessagesPlaceholder
from langchain_core.runnables.history import RunnableWithMessageHistory
from langchain_core.chat_history import InMemoryChatMessageHistory

# 1. Index documents
docs = WebBaseLoader('https://docs.langchain.com').load()
chunks = RecursiveCharacterTextSplitter(chunk_size=1000).split_documents(docs)
db = Chroma.from_documents(chunks, OpenAIEmbeddings())
retriever = db.as_retriever(search_kwargs={'k': 4})

llm = ChatOpenAI(model='gpt-4o', temperature=0)

# 2. Contextualise question with chat history
ctx_prompt = ChatPromptTemplate.from_messages([
    ('system', 'Given chat history, reformulate the user question standalone.'),
    MessagesPlaceholder('chat_history'), ('human', '{input}')
])
hist_retriever = create_history_aware_retriever(llm, retriever, ctx_prompt)

# 3. Answer prompt
ans_prompt = ChatPromptTemplate.from_messages([
    ('system', 'Answer using context only:\n{context}'),
    MessagesPlaceholder('chat_history'), ('human', '{input}')
])
qa_chain = create_stuff_documents_chain(llm, ans_prompt)
rag_chain = create_retrieval_chain(hist_retriever, qa_chain)

# 4. Add memory
store = {}
get_history = lambda sid: store.setdefault(sid, InMemoryChatMessageHistory())
chatbot = RunnableWithMessageHistory(rag_chain, get_history,
    input_messages_key='input', history_messages_key='chat_history',
    output_messages_key='answer')

cfg = {'configurable': {'session_id': 's1'}}
r1 = chatbot.invoke({'input': 'What is LangChain?'}, config=cfg)
r2 = chatbot.invoke({'input': 'What did I just ask about?'}, config=cfg)
print(r2['answer'])
```

Module 33 End-to-End Q&A ChatBot Gen AI App

Build a complete Streamlit Q&A chatbot app supporting both OpenAI (cloud) and Ollama (local) models — with environment management and a polished UI.

Lecture 1 — Setup

```
# .env
OPENAI_API_KEY=sk-...

pip install langchain langchain-openai langchain-community
pip install streamlit python-dotenv

# Project structure
app.py
.env
requirements.txt
```

Lecture 2 — OpenAI Application

```
# app.py (OpenAI variant)
import streamlit as st
from langchain_openai import ChatOpenAI
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser
from dotenv import load_dotenv
load_dotenv()

st.title('Q&A ChatBot')
model_name = st.selectbox('Model', ['gpt-4o', 'gpt-4o-mini', 'gpt-3.5-turbo'])
temperature = st.slider('Temperature', 0.0, 1.0, 0.7)

prompt = ChatPromptTemplate.from_messages([
    ('system', 'You are a helpful assistant.'),
    ('human', '{question}')
])
chain = prompt | ChatOpenAI(model=model_name, temperature=temperature) | StrOutputParser()

question = st.text_input('Ask a question:')
if st.button('Submit') and question:
    with st.spinner('Thinking...'):
        st.write(chain.invoke({'question': question}))
```

Lecture 3 — Ollama Open Source Model

```
# Switch to local Ollama model – no API cost
from langchain_community.llms import Ollama

model_name = st.selectbox('Model', ['llama3', 'phi3', 'gemma2'])
chain = prompt | Ollama(model=model_name) | StrOutputParser()
# All else identical – model-agnostic LCEL syntax
```

Module 34 RAG Document Q&A with Groq API and LLaMA 3

Full RAG pipeline using Groq's ultra-fast LLaMA 3 inference. Upload PDFs, embed with HuggingFace (free), retrieve with FAISS, answer with Groq.

Lecture 1 — Getting Started with ChatGroq RAG

```
pip install langchain langchain-groq langchain-community
pip install faiss-cpu pypdf sentence-transformers streamlit

# .env
GROQ_API_KEY=gsk...
```

Lecture 2 — End-to-End Groq RAG Application

```
import streamlit as st
from langchain_groq import ChatGroq
from langchain_community.document_loaders import PyPDFDirectoryLoader
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain_community.embeddings import HuggingFaceEmbeddings
from langchain_community.vectorstores import FAISS
from langchain_core.prompts import ChatPromptTemplate
from langchain.chains import create_retrieval_chain
from langchain.chains.combine_documents import create_stuff_documents_chain
from dotenv import load_dotenv
load_dotenv()

st.title('RAG Q&A - Groq + LLaMA 3')

llm = ChatGroq(model='llama3-8b-8192', temperature=0)

@st.cache_resource
def build_vectorstore():
    docs = PyPDFDirectoryLoader('pdfs/').load()
    chunks = RecursiveCharacterTextSplitter(chunk_size=1000).split_documents(docs)
    embed = HuggingFaceEmbeddings(model_name='all-MiniLM-L6-v2')
    return FAISS.from_documents(chunks, embed)

db = build_vectorstore()
retriever = db.as_retriever(search_kwargs={'k': 4})

prompt = ChatPromptTemplate.from_template(
    'Answer using only context:\n{context}\nQuestion: {input}'
)
chain = create_retrieval_chain(retriever, create_stuff_documents_chain(llm, prompt))

q = st.text_input('Ask about your PDFs:')
if q:
    result = chain.invoke({'input': q})
    st.write(result['answer'])
    with st.expander('Sources'):
        for doc in result['context']:
            st.write(doc.metadata['source'], '-', doc.page_content[:200])
```

Module 35 Conversational Q&A Chat with PDF ChatBot + History

Upload one or more PDFs via Streamlit, index them, and chat with multi-turn memory — answers reference specific pages and remember previous questions.

Lecture 1 — Setup and Loading PDFs

```
pip install langchain langchain-openai pypdf chromadb streamlit

# Upload PDFs in Streamlit
uploaded = st.file_uploader('Upload PDFs', type='pdf', accept_multiple_files=True)

import tempfile, os
from langchain_community.document_loaders import PyPDFLoader
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain_openai import OpenAIEmbeddings
from langchain_community.vectorstores import Chroma

if uploaded:
    all_docs = []
    for f in uploaded:
        with tempfile.NamedTemporaryFile(delete=False, suffix='.pdf') as tmp:
            tmp.write(f.read()); tmp_path = tmp.name
        all_docs.extend(PyPDFLoader(tmp_path).load())
    chunks = RecursiveCharacterTextSplitter(chunk_size=1000).split_documents(all_docs)
    db = Chroma.from_documents(chunks, OpenAIEmbeddings())
    st.success(f'Indexed {len(chunks)} chunks from {len(uploaded)} PDFs')
```

Lecture 2 — Conversational RAG with History

```
from langchain.chains import create_history_aware_retriever, create_retrieval_chain
from langchain.chains.combine_documents import create_stuff_documents_chain
from langchain_core.prompts import ChatPromptTemplate, MessagesPlaceholder
from langchain_core.runnables.history import RunnableWithMessageHistory
from langchain_core.chat_history import InMemoryChatMessageHistory
from langchain_openai import ChatOpenAI

llm = ChatOpenAI(model='gpt-4o', temperature=0)
retriever = db.as_retriever(search_kwargs={'k': 4})

ctx_q = ChatPromptTemplate.from_messages([
    ('system', 'Reformulate user question as standalone given history.'),
    MessagesPlaceholder('chat_history'), ('human', '{input}')
])
ans_q = ChatPromptTemplate.from_messages([
    ('system', 'Answer from context:\n{context}'),
    MessagesPlaceholder('chat_history'), ('human', '{input}')
])

rag = create_retrieval_chain(
    create_history_aware_retriever(llm, retriever, ctx_q),
    create_stuff_documents_chain(llm, ans_q)
)
store = {}
bot = RunnableWithMessageHistory(rag, lambda sid: store.setdefault(sid,
    InMemoryChatMessageHistory()),
    input_messages_key='input', history_messages_key='chat_history', output_messages_key='answer')

# Streamlit chat interface
if 'msgs' not in st.session_state: st.session_state.msgs = []
for m in st.session_state.msgs:
    st.chat_message(m['role']).write(m['content'])
if q := st.chat_input('Ask about your PDFs...'):
    st.session_state.msgs.append({'role': 'user', 'content': q})
    r = bot.invoke({'input': q}, config={'configurable': {'session_id': 'pdf_chat'}})
    st.session_state.msgs.append({'role': 'assistant', 'content': r['answer']})
    st.rerun()
```

Module 37 Chat with SQL DB — LangChain SQL Toolkit and Agent

Build a natural language interface to a SQL database. LangChain's SQL agent translates user questions into SQL queries, executes them, and returns human-readable answers.

Lecture 1 — Storing Data in SQLite

```
import sqlite3, pandas as pd

conn = sqlite3.connect('company.db')
df = pd.read_csv('employees.csv')
df.to_sql('employees', conn, if_exists='replace', index=False)
conn.commit(); conn.close()

# Verify
conn = sqlite3.connect('company.db')
print(pd.read_sql('SELECT * FROM employees LIMIT 5', conn))
```

Lectures 2 & 3 — SQL Database Engine

```
from langchain_community.utilities import SQLDatabase
from langchain_openai import ChatOpenAI

# Connect LangChain to the SQLite database
db = SQLDatabase.from_uri('sqlite:///company.db')
llm = ChatOpenAI(model='gpt-4o', temperature=0)

print(db.get_usable_table_names()) # ['employees', ...]
print(db.table_info)              # schema for context
```

Lecture 4 — SQL Agent

```
from langchain_community.agent_toolkits import create_sql_agent
from langchain_community.agent_toolkits.sql.toolkit import SQLDatabaseToolkit

toolkit = SQLDatabaseToolkit(db=db, llm=llm)

agent = create_sql_agent(
    llm=llm,
    toolkit=toolkit,
    verbose=True,
    agent_type='openai-tools'
)

# Natural language -> SQL -> result -> English answer
r = agent.invoke('How many employees earn more than 80000?')
print(r['output'])

r = agent.invoke('List the top 5 highest paid employees')
print(r['output'])

# Agent automatically:
# 1. Inspects schema 2. Writes SQL 3. Executes 4. Formats answer
```


Module 38 Text Summarization with LangChain

Three summarisation strategies in LangChain — Stuff (simple), Map-Reduce (parallel), and Refine (iterative) — each suited to different document sizes and quality requirements.

Lecture 1 — Basic Summarization with PromptTemplate

```
from langchain_openai import ChatOpenAI
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser

llm = ChatOpenAI(model='gpt-4o')
chain = ChatPromptTemplate.from_template(
    'Summarize the following text in 3 bullet points:\n{text}'
) | llm | StrOutputParser()

summary = chain.invoke({'text': long_document})
```

Lecture 2 — Stuff Summarization Chain

```
from langchain.chains.summarize import load_summarize_chain
from langchain_community.document_loaders import PyPDFLoader

docs = PyPDFLoader('report.pdf').load()

# Stuff: concatenate ALL docs into one prompt – best for short docs
stuff_chain = load_summarize_chain(llm, chain_type='stuff')
summary = stuff_chain.invoke({'input_documents': docs})
print(summary['output_text'])

# Custom stuff prompt
from langchain_core.prompts import PromptTemplate
custom = PromptTemplate.from_template(
    'Write a concise executive summary:\n\n{text}'
)
stuff_chain = load_summarize_chain(llm, chain_type='stuff', prompt=custom)
```

Lecture 3 — Map-Reduce and Refine Chains

```
# Map-Reduce: summarize each chunk independently, then combine
# Best for: long documents, parallelisable
mapreduce = load_summarize_chain(llm, chain_type='map_reduce')
summary = mapreduce.invoke({'input_documents': chunks})

# Refine: summarize first chunk, then iteratively update with each next chunk
# Best for: coherent narrative, sequential context matters
refine = load_summarize_chain(llm, chain_type='refine')
summary = refine.invoke({'input_documents': chunks})

# Comparison
# Stuff: best quality, limited by context window (~128K tokens max)
# Map-Reduce: scalable, parallel, slight quality loss from disconnected chunks
# Refine: high quality, sequential, slowest (one LLM call per chunk)
```

Module 39 Text to Math Problem Solver

Build an agent that takes natural language math problems, breaks them into steps using LLM reasoning, evaluates expressions with Python, and returns a step-by-step solution.

Lecture 1 — Getting Started with Text to Math App

```
pip install langchain langchain-openai numexpr streamlit

# Tools the agent will use
from langchain.tools import Tool
from langchain_experimental.tools import PythonREPLTool
import math, numexpr

def evaluate_math(expression: str) -> str:
    try:
        result = numexpr.evaluate(expression)
        return str(result)
    except Exception as e:
        return f'Error: {e}'

math_tool = Tool(
    name = 'Calculator',
    func = evaluate_math,
    description = 'Evaluates mathematical expressions. Input: valid math expression string.'
)

python_tool = PythonREPLTool() # runs Python code for complex maths
```

Lecture 2 — Creating Agent and Session State

```
import streamlit as st
from langchain_openai import ChatOpenAI
from langchain.agents import create_react_agent, AgentExecutor
from langchain import hub

llm = ChatOpenAI(model='gpt-4o', temperature=0)
tools = [math_tool, python_tool]
prompt = hub.pull('hwchase17/react') # standard ReAct prompt

agent = create_react_agent(llm, tools, prompt)
executor = AgentExecutor(agent=agent, tools=tools, verbose=True)

# Streamlit UI with session state
st.title('Math Problem Solver')
if 'history' not in st.session_state: st.session_state.history = []

problem = st.text_area('Enter a math problem:', height=100)
if st.button('Solve'):
    with st.spinner('Solving...'):
        result = executor.invoke({'input': problem})
        st.session_state.history.append({'q': problem, 'a': result['output']})
        st.success(result['output'])

if st.session_state.history:
    st.subheader('History')
    for item in reversed(st.session_state.history):
        st.write(f'Q: {item["q"]}')
        st.write(f'A: {item["a"]}')
    st.divider()
```

Module 40 HuggingFace and LangChain Integration

Use HuggingFace models (both hosted via Inference API and local via pipeline) directly in LangChain chains — free, open-source alternative to OpenAI.

Lecture 1 — Getting Started with HuggingFace Models

```

pip install langchain langchain-huggingface transformers torch

# .env
HUGGINGFACEHUB_API_TOKEN=hf_...

# — HuggingFace Inference API (hosted, free tier) —————
from langchain_huggingface import HuggingFaceEndpoint

llm = HuggingFaceEndpoint(
    repo_id      = 'mistralai/Mistral-7B-Instruct-v0.2',
    temperature  = 0.7,
    max_new_tokens=256
)
print(llm.invoke('What is LangChain?'))

# — Local pipeline (runs on your machine) —————
from langchain_huggingface import HuggingFacePipeline
from transformers import pipeline

pipe = pipeline('text-generation', model='microsoft/phi-2',
               max_new_tokens=200, device=-1) # -1=CPU, 0=GPU
local_llm = HuggingFacePipeline(pipeline=pipe)
print(local_llm.invoke('Explain neural networks briefly.'))

```

Lecture 2 — ChatPromptTemplate with HuggingFace

```

from langchain_huggingface import HuggingFaceEndpoint
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser

llm = HuggingFaceEndpoint(repo_id='mistralai/Mistral-7B-Instruct-v0.2')

# Note: some HF models expect specific instruction format
prompt = ChatPromptTemplate.from_messages([
    ('system', 'You are a helpful assistant.'),
    ('human', '{question}')
])

chain = prompt | llm | StrOutputParser()
print(chain.invoke({'question': 'What is the difference between LSTM and GRU?'}))

# HuggingFace Embeddings (free, local)
from langchain_huggingface import HuggingFaceEmbeddings
embeddings = HuggingFaceEmbeddings(model_name='all-MiniLM-L6-v2')
vec = embeddings.embed_query('LangChain and HuggingFace')
print(f'Embedding dim: {len(vec)}') # 384

```

Module 41 PDF Query RAG with LangChain and AstraDB

Use DataStax AstraDB — a managed Cassandra-backed vector database — as the vector store for a PDF RAG application. AstraDB is free-tier available and requires no local infrastructure.

Lecture 1 — Getting Started with AstraDB

```
pip install langchain-astradb astrapy pypdf langchain-openai

# Get credentials from astra.datastax.com:
# ASTRA_DB_APPLICATION_TOKEN=AstraCS:...
# ASTRA_DB_API_ENDPOINT=https://UUID-REGION.apps.astra.datastax.com

from langchain_astradb import AstraDBVectorStore
from langchain_openai import OpenAIEmbeddings

vstore = AstraDBVectorStore(
    embedding = OpenAIEmbeddings(),
    collection_name = 'pdf_rag',
    api_endpoint = os.environ['ASTRA_DB_API_ENDPOINT'],
    token = os.environ['ASTRA_DB_APPLICATION_TOKEN'],
)
print('AstraDB vector store ready')
```

Lecture 2 — Creating RAG App with AstraDB

```
from langchain_community.document_loaders import PyPDFLoader
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain_openai import ChatOpenAI
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.runnables import RunnablePassthrough
from langchain_core.output_parsers import StrOutputParser

# Index PDFs into AstraDB
docs = PyPDFLoader('data/research.pdf').load()
chunks = RecursiveCharacterTextSplitter(chunk_size=1000).split_documents(docs)
vstore.add_documents(chunks)
print(f'Indexed {len(chunks)} chunks to AstraDB')

# RAG chain
retriever = vstore.as_retriever(search_kwargs={'k': 4})
llm = ChatOpenAI(model='gpt-4o', temperature=0)
prompt = ChatPromptTemplate.from_template(
    'Answer using context only:\n{context}\n\nQuestion: {question}'
)

chain = (
    {'context': retriever, 'question': RunnablePassthrough()}
    | prompt | llm | StrOutputParser()
)
print(chain.invoke('What are the main findings of the research?'))

# AstraDB persists across sessions – no rebuild needed
# vstore.delete_collection() to reset
```

Module 42 Working with CodeLlama via Ollama

Run Meta's CodeLlama locally with Ollama for code generation, explanation, debugging, and completion — all free, private, and offline.

Lecture 1 — Getting Started with CodeLlama

```
# Pull CodeLlama models via Ollama CLI
ollama pull codellama          # 7B base, ~4 GB
ollama pull codellama:13b      # 13B, ~8 GB – better quality
ollama pull codellama:code     # code-only variant

# Test in terminal
ollama run codellama 'Write a Python function to check if a number is prime'

# Python – LangChain integration
from langchain_community.llms import Ollama

llm = Ollama(model='codellama')
print(llm.invoke('Write a Python quicksort function.'))
```

Lecture 2 — Creating the CodeLlama Application

```
import streamlit as st
from langchain_community.llms import Ollama
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser

st.title('CodeLlama Code Assistant')

task = st.selectbox('Task', ['Generate', 'Explain', 'Debug', 'Convert'])
language = st.selectbox('Language', ['Python', 'JavaScript', 'SQL', 'Java'])

prompts = {
    'Generate': f'Write a {{language}} function for: {{code}}',
    'Explain': 'Explain this code step by step:\n{{code}}',
    'Debug': 'Find and fix bugs in this code, explain each fix:\n{{code}}',
    'Convert': f'Convert this code to {{language}}:\n{{code}}',
}

llm = Ollama(model='codellama')
chain = ChatPromptTemplate.from_template(prompts[task]) | llm | StrOutputParser()

code_input = st.text_area('Enter your code or description:', height=200)
if st.button('Run') and code_input:
    with st.spinner('CodeLlama thinking...'):
        result = chain.invoke({'code': code_input, 'language': language})
    st.code(result, language=language.lower())
```

Module 43 Deployment of Gen AI Apps

Deploy your Streamlit Gen AI apps to the cloud — either Streamlit Cloud (easiest) or HuggingFace Spaces (free, GPU-available). Both are free-tier and take under 10 minutes.

Lecture 1 — Deployment in Streamlit Cloud

```
# Step 1: Prepare your repository
# File structure needed:
# myapp/
# ├── app.py           <- must be named app.py
# ├── requirements.txt  <- all pip dependencies
# └── .gitignore        <- include .env

# requirements.txt example
langchain==0.2.0
langchain-openai==0.1.0
langchain-community==0.2.0
streamlit==1.35.0
python-dotenv==1.0.0

# Step 2: Push to GitHub
git init && git add . && git commit -m 'deploy'
git remote add origin https://github.com/USER/REPO.git
git push origin main

# Step 3: Deploy
# 1. Go to share.streamlit.io
# 2. Sign in with GitHub
# 3. New app -> select repo, branch (main), file (app.py)
# 4. Advanced settings -> add secrets (API keys)
#     OPENAI_API_KEY = 'sk-...'
#     GROQ_API_KEY   = 'gsk-...'
# 5. Deploy -> live at https://user-appname-xxx.streamlit.app

# In app.py: access secrets via st.secrets
import os
os.environ['OPENAI_API_KEY'] = st.secrets['OPENAI_API_KEY']
```

Lecture 2 — Deployment in HuggingFace Spaces

```
# HuggingFace Spaces: free hosting with optional GPU access

# Step 1: Create a Space
# 1. Go to huggingface.co/new-space
# 2. Choose SDK: Streamlit (or Gradio)
# 3. Set hardware: CPU Basic (free) or T4 GPU (paid)

# Step 2: File structure
# myspace/
# ├── app.py
# ├── requirements.txt
# └── README.md <- Space card (optional)

# Step 3: Add secrets
# Space Settings -> Variables and Secrets
# Name: OPENAI_API_KEY Value: sk-...

# Step 4: Push via git
git clone https://huggingface.co/spaces/USERNAME/SPACE_NAME
# copy your files in
git add . && git commit -m 'add app'
git push

# Access secrets in code (same as environment variables)
import os
api_key = os.environ.get('OPENAI_API_KEY')

# App live at: https://huggingface.co/spaces/USERNAME/SPACE_NAME
```

Platform	Free Tier GPU Best For
Streamlit Cloud	Yes — unlimited public apps No Quick demos, LLM apps without heavy compute
HuggingFace Spaces	Yes — CPU basic Yes (paid) ML apps, models requiring GPU inference
Render / Railway	Limited free No FastAPI backends, REST API deployment